Straight-Line Drag Run Detection

Validation_Aero_Test

January 13, 2026

Overview

This document explains the algorithm implemented in `Code/common.py` as `segment_drag_runs`. It detects straight-line drag runs that:

- begin at a local maximum (peak) of speed $v(t)$,
- continue while steering remains small and speed does not increase substantially,
- end when braking starts, re-acceleration occurs, turning exceeds a threshold, or speed becomes very low.

Inputs

- Time vector t (seconds).
- GPS speed magnitude v (m/s): `GPS1_VEL_MAG_ms`.
- Steering angle θ (deg): `ICU_SteeringAngle_deg`.
- Optional brake pressures (bar): front `ICU_Brakpresf_bar`, rear `ICU_Brakpresr_bar`.

Preprocessing

- Estimate median sample period Δt from t.
- Smooth v and θ by centered moving averages with window sizes derived from time-based windows.
- Compute $\dot{v} = \frac{dv}{dt}$ via `numpy.gradient` on the smoothed speed.
- If brakes are present, combine to $b(t) = \max(b_f, b_r)$ and lightly smooth.

Peak Selection

A sample i is a candidate peak if:

- $v_s[i]$ is a local maximum vs. neighbors,
- $v_s[i] \geq$ `min_peak_speed`,
- Prominence $p = v_s[i] - P_{80}(\{v_s\}_{i-\omega:i+\omega}) \geq$ `peak_prominence`, where P_{80} is the 80th percentile and ω is the peak window in samples.

Forward Extension and End Conditions

For each peak index p, extend forward sample-by-sample to find an end index e where any condition triggers:

- **Stop**: $v_s[i] \leq$ `stop_speed_th`.
- **Turning**: $|\theta_s[i]| >$ `steer_thresh` sustained for `turn_window_s`.
- **Braking**: $b[i] >$ `brake_thresh` or $\frac{db}{dt} >$ `brake_spike_thresh`.
- **Re-acceleration**: Over an acceleration window, if $v_s[j] - v_s[i] >$ `reaccel_eps` *and* mean $\dot{v}$ in that window exceeds a small positive threshold.

Quality Gates

After determining (p, e):

- **Straightness**: fraction of samples in $[p, e)$ where $|\theta_s| \leq$ `steer_thresh` must be $\geq$ `straight_ratio`.
- **Minimum duration**: $t[e-1] - t[p] \geq$ `min_dur_s`.

Accepted runs are returned as half-open intervals (p, e).

Default Parameters

`steer_thresh`	10 deg
`straight_ratio`	0.80
`min_peak_speed`	1.0 m/s
`peak_prominence`	0.1 m/s
`reaccel_eps`	0.2 m/s
`accel_window_s`	0.5 s
`brake_thresh`	0.05 bar
`brake_spike_thresh`	0.1 bar/s
`turn_window_s`	0.2 s
`min_dur_s`	0.6 s
`stop_speed_th`	0.2 m/s

Pseudocode

Function `segment_drag_runs`$(t, v, \theta, b_f, b_r, cfg)$

```
# Preprocess
dt_med = median(diff(t) > 0)
peak_win  = round(cfg.peak_window_s / dt_med)
accel_win = round(cfg.accel_window_s / dt_med)
turn_win  = round(cfg.turn_window_s / dt_med)
vs = smooth(v, window = max(5, 2*peak_win+1))
ths = smooth(theta, window = max(5, 2*turn_win+1)) if theta else None
bt = smooth(max(b_f, b_r), window = 5) if brakes else None
dv = gradient(vs, t)

# Find candidate peaks
```

```
peaks = []
for i in 1..n-2:
  if vs[i] >= vs[i-1] and vs[i] >= vs[i+1]:
    win = vs[i-peak_win : i+peak_win]
    prom = vs[i] - percentile(win, 80)
    if vs[i] >= cfg.min_peak_speed and prom >= cfg.peak_prominence:
      peaks.append(i)

# Extend segments from peaks
intervals = []
last_end = -1
for p in peaks:
  if p <= last_end: continue
  start = p; i = p+1; turn_count = 0
  while i < n:
    if vs[i] <= cfg.stop_speed_th: i += 1; break
    if ths and abs(ths[i]) > cfg.steer_thresh: turn_count += 1
    else: turn_count = 0
    if turn_count >= turn_win: break
    if bt:
      if bt[i] > cfg.brake_thresh: break
      if (bt[i] - bt[i-1]) / dt_med > cfg.brake_spike_thresh: break
    j = min(n-1, i + accel_win)
    dv_mean = mean(dv[i:j]) if j > i else 0
    if (vs[j] - vs[i] > cfg.reaccel_eps) and (dv_mean > 0.02): break
    i += 1
  end = max(i, start+1)
  if ths:
    straight = mean(abs(ths[start:end]) <= cfg.steer_thresh)
    if straight < cfg.straight_ratio: continue
  if t[end-1] - t[start] < cfg.min_dur_s: continue
  intervals.append((start, end)); last_end = end
return intervals
```

Notes

- All thresholds are configurable via `config` or CLI flags in `Code/export_drag_runs.py`.
- Smoothing reduces noise; window sizes are tied to time windows so behavior scales with sample rate.
- Straightness is enforced statistically over the segment, not at every sample, to avoid discarding good runs due to brief steering spikes.